NAME : SALVA SOUMYA

REGISTER NUMBER : 23MID0061

Data Preprocessing and ML model pipeline for loan prediction

Question

Loan Eligibility Dataset

13 columns, 600+ Records

1. Investigate the Dataset and document your findings.
2. Perform necessary Pre-Processing, add any column as required for your analysis, train a model for predicting Loan Eligibility, save the model file and test its accuracy.
3. Report the accuracy and take steps to improve further on accuracy.
4. Document the measures that you take to prevent overfitting.

Importing the libraries

In [32]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classificat
```

Loading the dataset

In [19]:
```
#  Loading the dataset
df = pd.read_csv("/home/matlab/Downloads/Loan Eligibility Prediction.csv"
df
```

Out[19]:

	Customer_ID	Gender	Married	Dependents	Education	Self_Employed	Applican
0	569	Female	No	0	Graduate	No	
1	15	Male	Yes	2	Graduate	No	
2	95	Male	No	0	Not Graduate	No	
3	134	Male	Yes	0	Graduate	Yes	
4	556	Male	Yes	1	Graduate	No	
...	...	...	...	...	...	...	
609	480	Male	Yes	2	Graduate	No	
610	525	Male	No	0	Graduate	No	
611	551	Male	Yes	2	Graduate	Yes	
612	552	Male	Yes	1	Not Graduate	No	
613	606	Male	Yes	0	Not Graduate	No	

614 rows × 13 columns

Exploration of the dataset

In [7]:
```
print(df.head())      #print the frst 5 rows of teh dataset
```

```
   Customer_ID  Gender Married  Dependents     Education Self_Employed  \
0          569  Female      No           0      Graduate            No
1           15    Male     Yes           2      Graduate            No
2           95    Male      No           0  Not Graduate            No
3          134    Male     Yes           0      Graduate           Yes
4          556    Male     Yes           1      Graduate            No

   Applicant_Income  Coapplicant_Income  Loan_Amount  Loan_Amount_Term  \
0              2378                 0.0            9               360
1              1299              1086.0           17               120
2              3620                 0.0           25               120
3              3459                 0.0           25               120
4              5468              1032.0           26               360

   Credit_History Property_Area Loan_Status
0               1         Urban           N
1               1         Urban           Y
2               1     Semiurban           Y
3               1     Semiurban           Y
4               1     Semiurban           Y
```

In [15]:
```
print(df.tail())      #prints last 5 rows of the dataset
```

```
     Customer_ID Gender Married  Dependents     Education Self_Employed  \
609          480   Male     Yes           2      Graduate            No
610          525   Male      No           0      Graduate            No
611          551   Male     Yes           2      Graduate           Yes
612          552   Male     Yes           1  Not Graduate            No
613          606   Male     Yes           0  Not Graduate            No

     Applicant_Income  Coapplicant_Income  Loan_Amount  Loan_Amount_Term
\
609              2947              1603.0           10               360
610              4680              2087.0           15               360
611              6633                 0.0           25               360
612              2492              2375.0           30               360
613              2400              3800.0           25               180

     Credit_History Property_Area Loan_Status
609               1         Urban           N
610               1     Semiurban           N
611               0         Rural           N
612               1         Rural           Y
613               1         Urban           N
```

In [8]:
```
print(df.shape)       #prints the number of rows and columns in the datse
```

```
(614, 13)
```

In [9]:
```
print(df.columns)    #prints all the columns in the dataset
```

```
Index(['Customer_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'Applicant_Income', 'Coapplicant_Income',
       'Loan_Amount', 'Loan_Amount_Term', 'Credit_History', 'Property_Are
a',
       'Loan_Status'],
      dtype='object')
```

In [10]:
```
print(df.isnull().sum())     # prints the sum of missing values in each c
```

```
Customer_ID           0
Gender                0
Married               0
Dependents            0
Education             0
Self_Employed         0
Applicant_Income      0
Coapplicant_Income    0
Loan_Amount           0
Loan_Amount_Term      0
Credit_History        0
Property_Area         0
Loan_Status           0
dtype: int64
```

In [16]:
```
print(df.isnull().sum().sum())  #prints the sum of missing values in the
```

```
0
```

Drop the duplicates in the dataset

In [18]:
```
df = df.drop_duplicates()
df
```

Out[18]:

	Customer_ID	Gender	Married	Dependents	Education	Self_Employed	Applican
0	569	Female	No	0	Graduate	No	
1	15	Male	Yes	2	Graduate	No	
2	95	Male	No	0	Not Graduate	No	
3	134	Male	Yes	0	Graduate	Yes	
4	556	Male	Yes	1	Graduate	No	
...	...	...	...	...	...	...	
609	480	Male	Yes	2	Graduate	No	
610	525	Male	No	0	Graduate	No	
611	551	Male	Yes	2	Graduate	Yes	
612	552	Male	Yes	1	Not Graduate	No	
613	606	Male	Yes	0	Not Graduate	No	

614 rows × 13 columns

Ml model pipeline

In [12]:
```
# Setting the value for the target

target = "Loan_Status"

# Removing rows with missing target

df = df.dropna(subset=[target])

# Removing ID column

df = df.drop(columns=["Loan_ID"], errors="ignore")
print(df)
```

```
     Customer_ID  Gender Married  Dependents     Education Self_Employed
\
0            569  Female      No           0      Graduate            No
1             15    Male     Yes           2      Graduate            No
2             95    Male      No           0  Not Graduate            No
3            134    Male     Yes           0      Graduate           Yes
4            556    Male     Yes           1      Graduate            No
..           ...     ...     ...         ...           ...           ...
609          480    Male     Yes           2      Graduate            No
610          525    Male      No           0      Graduate            No
611          551    Male     Yes           2      Graduate           Yes
612          552    Male     Yes           1  Not Graduate            No
613          606    Male     Yes           0  Not Graduate            No

     Applicant_Income  Coapplicant_Income  Loan_Amount  Loan_Amount_Term
\
0                2378                 0.0            9               360
1                1299              1086.0           17               120
2                3620                 0.0           25               120
3                3459                 0.0           25               120
4                5468              1032.0           26               360
..                ...                 ...          ...               ...
609              2947              1603.0           10               360
610              4680              2087.0           15               360
611              6633                 0.0           25               360
612              2492              2375.0           30               360
613              2400              3800.0           25               180

     Credit_History Property_Area Loan_Status
0                 1         Urban           N
1                 1         Urban           Y
2                 1     Semiurban           Y
3                 1     Semiurban           Y
4                 1     Semiurban           Y
..              ...           ...         ...
609               1         Urban           N
610               1     Semiurban           N
611               0         Rural           N
612               1         Rural           Y
613               1         Urban           N

[614 rows x 13 columns]
```

Storing the values of X and Y

In [ ]:
```
X = df.drop(columns=[target])   #stpring the columns other than target in
y = df[target]         #storing the traget in y
```

Mean and mode imputation for missing values

In [23]:
```
for col in X.columns:
    if X[col].dtype == "object":
        X[col] = X[col].fillna(X[col].mode()[0])       #for categorical c
```

```
    else:
        X[col] = X[col].fillna(X[col].median())        #for numerical col
```

Encoding the categorical columns using LabelEncoder()

In [27]:
```
for col in X.columns:
    if X[col].dtype == "object":
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))
    if y.dtype == "object":
        le = LabelEncoder()
        y = le.fit_transform(y.astype(str))
```

Train-Test split for splitting training and testing data

In [28]:
```
#80-20 split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Training the model using LogisticRegression

In [29]:
```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
/home/matlab/.local/lib/python3.10/site-packages/sklearn/linear_model/_log
istic.py:473: ConvergenceWarning: lbfgs failed to converge after 1000 iter
ation(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100
0).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-reg
ression
  n_iter_i = _check_optimize_result(
```

Evaluation metrics of the model

In [30]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("Classification Report:\n",
      classification_report(y_test, y_pred, zero_division=0))
```

```
Accuracy: 0.8211382113821138
Confusion Matrix:
 [[19 21]
 [ 1 82]]
Classification Report:
               precision    recall  f1-score   support

           0       0.95      0.47      0.63        40
           1       0.80      0.99      0.88        83

    accuracy                           0.82       123
   macro avg       0.87      0.73      0.76       123
weighted avg       0.85      0.82      0.80       123
```

Training the model using Descison Tree

Decison tree

In [47]:
```
#train the model
model_2 = DecisionTreeClassifier(random_state=42)
model_2.fit(X_train, y_train)
y_pred = model_2.predict(X_test)
```

Evaluation metrics of decision tree

In [48]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report( Y_test, y_pred))
```

```
Accuracy: 0.7235772357723578
Confusion Matrix:
[[24 16]
 [18 65]]
Classification Report:
              precision    recall  f1-score   support

           0       0.57      0.60      0.59        40
           1       0.80      0.78      0.79        83

    accuracy                           0.72       123
   macro avg       0.69      0.69      0.69       123
weighted avg       0.73      0.72      0.73       123
```

4. Document the measures that you take to prevent overfitting.

1.check any class imabalance in the datset before the model training

2.if the model has bias then improve the data by normalising the data without much class imabalance

3.using k-fold stratified models

4.analyse the data by necessary preprocessing (Data Wrangling)

In [ ]: